

Add a New System Call with args to copy file in kernel

Target

1. Add a new system call with arguments into the linux kernel on Kylin desktop OS v10
2. The new system call will return all processes information to user mode
 - i. 实现内核中文件拷贝、并返回用户态
 - ii. see the last section:(copy srcfile to desfile in new sys call in kernel)

Tools

Install GCC Software Colletion

```
sudo apt-get install build-essential
```

How to use GCC

- [gcc and make](#)

How to do

step 1:

in include/linux/syscalls.h

add the defintion of the new system call:

```
asmlinkage long sys_alcall(int cmd, char* buf);
```

step 2:

in kernel/sys.c

Please modify the implementation, because it is just a framework.

```
SYSCALL_DEFINE2(alcall,int,cmd,char*,buf)
{
    struct task_struct *p;
    printk("Hello new system call alcall (%d,%x)!\n",cmd,buf);
    print("%-20s %-6s %-6s\n","Name","Pid","Stat");
    for (p = &init_task; (p = next_task(p)) != &init_task;)
        printk("%-20s %-6d %-6ld\n",p->comm,p->pid,p->state);
    return 0;
}
```

functions in kernel:

```
copy_to_user
copy_from_user
sprintf
strcpy
strcat
...
```

step 3:

in arch/x86/entry/syscalls/syscall_64.tbl

add:

```
4??      common  alcall      __x64_sys_alcall
```

step 4:

re-configure the kernel

download the configure file in the current project:

(optional) linux_config

in the directory of the kernel source code:

```
cp linux_config_ref20201203 .config
make oldconfig
make gconfig
```

recompile the kernel

```
make clean
make -j5
sudo make modules_install
sudo make install
```

reboot with the new kernel

reboot

step 5:

write a test program in user mode

see the previous lab.

copy srcfile to desfile in new sys call in kernel

- 相关头文件以及数据结构

```
linux/fs.h, asm/uaccess.h
strcut file*
get_fs()
filp_open(), vfs_read(),vfs_write(), filp_close()
```

- 相关参考代码

```
struct file *file = NULL;
mm_segment_t old_fs;
char buf[128] = "123456";
file = filp_open("/data/test.txt", O_RDWR | O_APPEND | O_CREAT, 0644);
if (IS_ERR(file)) {
return 0;
}
old_fs = get_fs();
set_fs(KERNEL_DS);
vfs_write(file, buf, sizeof(buf), 0);
set_fs(old_fs);
filp_close(file, NULL);
return 0;
```